

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 04 -

DESENVOLVIMENTO DE APIS RESTFUL COM SPRING MVC

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EXEMPLO

O QUE VEM POR AÍ?

Nesta aula, vamos aprender a criar uma API REST utilizando o framework Spring MVC. Começaremos pela construção dos componentes essenciais de uma aplicação Spring, como controllers, services e repositories. Além disso, utilizaremos bibliotecas importantes, como o Spring Data, para facilitar a manipulação de dados, e o Spring Validation, para garantir que os dados sejam sempre consistentes e válidos.

Também abordaremos tópicos como o retorno dos códigos de status HTTP corretos e a implementação de tratativas de erros de maneira eficaz. Para tornar o aprendizado mais prático, utilizaremos o banco de dados em memória H2, que permite uma rápida prototipação e testes da nossa API. Ao final da aula, você poderá criar APIs REST robustas e bem estruturadas, essenciais para o desenvolvimento de aplicações modernas e escaláveis.

HANDS ON

Nesta aula prática, mergulharemos na criação de uma API REST com Spring MVC. Iniciaremos configurando nosso projeto Spring Boot e exploraremos as principais anotações, como `@RestController`, `@RequestMapping` e `@GetMapping`, essenciais para definir nossos endpoints. Com a anotação `@RestController`, transformamos nossa classe em um controlador REST, enquanto `@RequestMapping` e `@GetMapping` nos permitem mapear as requisições HTTP para métodos específicos.

Para a camada de serviço, utilizaremos a anotação `@Service`, que indica que a classe fornece serviços de negócio. Já a camada de repositório será configurada com a anotação `@Repository`, integrando o Spring Data para facilitar operações de banco de dados. Vamos utilizar o banco de dados em memória H2, configurando-o no arquivo `application.properties` com as propriedades `spring.datasource.url`, `spring.datasource.username` e `spring.datasource.password`, permitindo uma configuração rápida e eficiente.

A validação dos dados será realizada com a anotação `@Valid`, assegurando que os dados de entrada estejam corretos antes de processá-los. Abordaremos também o tratamento de exceções com a anotação `@ExceptionHandler`, garantindo que os erros sejam gerenciados de maneira adequada e informativa. Cada passo será ilustrado com exemplos de código, proporcionando uma compreensão clara e prática de como construir uma API REST completa e funcional.

SAIBA MAIS

Anteriormente aprendemos sobre API e, de forma prática, construímos uma API aplicando a arquitetura REST. Mas, como vimos, existem outros tipos de API, como SOAP, RPC e Websocket. Vamos explorar um pouco mais sobre elas, iniciando com SOAP.

Simple Object Access Protocol (SOAP)

O Simple Object Access Protocol (SOAP) é um protocolo de comunicação desenvolvido pela W3C (World Wide Web Consortium) que permite a troca de informações estruturadas em um ambiente distribuído e descentralizado. Utilizando XML (Extensible Markup Language) como seu formato de mensagem, o SOAP foi projetado para ser independente de plataforma e linguagem de programação, permitindo assim que diferentes sistemas possam se comunicar de maneira eficaz e segura.

Uma mensagem SOAP é composta por três partes principais. O envelope é o elemento raiz da mensagem, que define o início e o fim da mensagem SOAP. Ele encapsula todas as demais partes da mensagem. O cabeçalho, opcional, pode conter informações como autenticação, transações e outros dados de controle que não são necessariamente parte dos dados da aplicação. Já o corpo contém a mensagem propriamente dita, que é o payload da comunicação. Aqui residem os dados que estão sendo transmitidos entre o cliente e o servidor.

O SOAP opera sobre protocolos de transporte como HTTP, SMTP, TCP, entre outros, sendo o HTTP o mais comumente utilizado. A flexibilidade do SOAP em operar sobre diferentes protocolos de transporte o torna uma escolha robusta para diversas aplicações, incluindo aquelas que exigem alta segurança e confiabilidade.

Uma das grandes vantagens do SOAP é sua independência de plataforma. Por ser baseado em XML, ele permite a comunicação entre sistemas desenvolvidos em diferentes linguagens de programação. Além disso, com suporte nativo para WS-Security, o SOAP oferece recursos avançados de segurança, incluindo autenticação, integridade e confidencialidade. O protocolo também suporta transações ACID, o que é crucial para aplicações empresariais que requerem operações transacionais seguras e confiáveis. A arquitetura do SOAP permite a adição de funcionalidades sem

a necessidade de alterar os serviços já existentes, graças ao seu cabeçalho extensível.

Entretanto, o SOAP apresenta algumas desvantagens. A configuração e manutenção de serviços SOAP pode ser complexa devido ao seu extenso uso de XML. Além disso, esse uso intensivo de XML e a necessidade de parsing podem resultar em um desempenho inferior quando comparado a outros protocolos mais leves, como REST. As mensagens SOAP tendem a ser mais verbosas, o que pode resultar em maior consumo de largura de banda e maior tempo de processamento.

Enquanto o SOAP é um protocolo rígido e fortemente tipado, o REST (Representational State Transfer) é um estilo arquitetônico mais flexível e leve, que utiliza os métodos HTTP para comunicação. O REST é frequentemente preferido para aplicações web devido à sua simplicidade e melhor desempenho, embora o SOAP continue a ser uma escolha popular em cenários que exigem maior segurança e complexidade.

O SOAP é um protocolo poderoso e versátil para a comunicação entre sistemas distribuídos. Apesar de suas desvantagens em termos de complexidade e desempenho, suas vantagens em termos de segurança, transacionalidade e extensibilidade o tornam uma escolha valiosa para muitas aplicações empresariais. Compreender o funcionamento e as características do SOAP é fundamental para qualquer pessoa desenvolvedora que esteja construindo soluções de integração entre diferentes sistemas. Por mais que não seja muito utilizado em projetos criados atualmente, ainda o encontramos muito no mercado.

Remote Procedure Call (RPC)

Outra API que podemos encontrar no dia a dia de trabalho é a RPC (Remote Procedure Call). RPC é um protocolo que permite a um programa executar uma função ou procedimento em outro endereço de espaço de memória, como se fosse uma chamada local. Essencialmente, RPC abstrai a comunicação de rede ao ponto de parecer que todas as chamadas são locais, simplificando a programação distribuída. Quando uma função é chamada, o cliente serializa os parâmetros e os envia ao servidor, que então desserializa, processa a solicitação e retorna o resultado ao cliente.

RPC é conhecido por sua simplicidade e eficiência. Sua estrutura é bastante direta: o cliente chama um procedimento, passa os parâmetros necessários e aguarda a resposta. Por não depender de uma linguagem específica, RPC pode ser implementado em várias linguagens de programação. Contudo, há desafios inerentes à sua operação, como a gestão de falhas na rede e a garantia de que a comunicação seja segura. A falta de padronização pode levar a problemas de compatibilidade entre diferentes implementações de RPC.

Um avanço significativo do RPC é o gRPC, desenvolvido pelo Google. O gRPC é um framework moderno que utiliza o HTTP/2 para transporte, Protobuf para serialização de dados e oferece suporte a uma variedade de linguagens de programação. A principal vantagem do gRPC é sua eficiência e desempenho, especialmente em sistemas de microsserviços. O uso do HTTP/2 proporciona multiplexação de múltiplas chamadas em uma única conexão, compressão de cabeçalhos e comunicação bidirecional por meio de streams, tudo isso resultando em uma comunicação mais rápida e eficiente.

Além disso, o gRPC simplifica a definição de serviços usando arquivos de definição de interface (IDL), que descrevem a estrutura das mensagens e serviços de maneira independente da linguagem de programação. Isso facilita a geração automática de código cliente e servidor em várias linguagens, promovendo a interoperabilidade entre diferentes sistemas.

Apesar das muitas vantagens, tanto RPC quanto gRPC têm suas desvantagens. RPC tradicional pode ser limitado em termos de interoperabilidade e flexibilidade, especialmente quando comparado a tecnologias mais modernas como RESTful APIs. O gRPC, por outro lado, embora mais avançado, pode ser mais complexo de configurar e gerenciar devido ao uso do Protobuf e HTTP/2. Além disso, a adoção do gRPC pode ser limitada pela necessidade de suporte ao HTTP/2, que nem sempre está disponível em todas as infraestruturas.

WebSocket

Outra API que podemos utilizar no nosso “canivete suíço” é o WebSocket. WebSocket é um protocolo de comunicação que permite uma interação bidirecional em tempo real entre um cliente e um servidor sobre uma única conexão TCP. Diferente

do tradicional HTTP, que segue um modelo de requisição-resposta, o WebSocket permite que dados sejam enviados e recebidos de forma contínua, sem a necessidade de reestabelecer a conexão para cada comunicação. Isso é especialmente útil para aplicações que exigem atualizações em tempo real, como chats online, jogos multiplayer e dashboards financeiros.

O funcionamento do WebSocket começa com um handshake HTTP tradicional, onde o cliente solicita uma conexão WebSocket ao servidor. Se o servidor aceita a solicitação, a conexão é atualizada do HTTP para o WebSocket e, a partir daí, a comunicação é full-duplex. Isso significa que tanto o cliente quanto o servidor podem enviar mensagens a qualquer momento, proporcionando uma interação dinâmica e eficiente. As mensagens são trocadas em um formato binário ou texto, permitindo a transmissão de diferentes tipos de dados de forma flexível.

Uma das principais vantagens do WebSocket é sua eficiência em termos de latência e uso de recursos. Ao manter uma conexão persistente, o WebSocket elimina a sobrecarga de abrir e fechar conexões repetidamente, resultando em uma comunicação mais rápida e responsiva. Além disso, o WebSocket é suportado nativamente pela maioria dos navegadores modernos e é compatível com uma ampla gama de linguagens de programação e frameworks, tornando-o uma escolha versátil para pessoas desenvolvedoras.

No entanto, a utilização do WebSocket vem com seus desafios. A gestão de conexões persistentes pode ser mais complexa, especialmente em cenários com muitos usuários simultâneos. Questões de segurança, como ataques de negação de serviço (DoS) e a necessidade de autenticação adequada, também precisam ser cuidadosamente abordadas. Além disso, o WebSocket pode não ser a melhor escolha para todas as aplicações; por exemplo, para comunicações simples ou onde a escalabilidade é uma preocupação maior, as requisições HTTP tradicionais ou outras tecnologias como HTTP/2 podem ser mais apropriadas.

Comparado a outras tecnologias de comunicação em tempo real, como Long Polling ou Server-Sent Events (SSE), o WebSocket oferece uma solução mais robusta e eficiente. Long Polling tenta simular uma comunicação em tempo real mantendo a conexão HTTP aberta até que uma nova informação esteja disponível, mas isso pode resultar em latência e uso ineficiente de recursos. SSE permite que o servidor envie dados para o cliente de forma unidirecional, o que é útil para certos tipos de

aplicações, mas não oferece a interatividade bidirecional que o WebSocket proporciona.

GraphQL

Para finalizar a nossa aula, trago uma carta na manga que pode nos ajudar a construir ou reconstruir uma solução: o GraphQL.

GraphQL é uma linguagem de consulta para APIs e um runtime para atender a essas consultas com seus dados existentes. Desenvolvida pelo Facebook em 2012 e aberta ao público em 2015, a principal característica do GraphQL é permitir que os clientes solicitem exatamente os dados de que precisam, nem mais, nem menos. Isso contrasta com as APIs REST tradicionais, onde cada endpoint retorna uma estrutura de dados fixa, muitas vezes resultando em overfetching ou underfetching de dados.

A operação de uma API GraphQL começa com o cliente definindo uma consulta que especifica exatamente os campos e objetos necessários. Em resposta, o servidor GraphQL processa essa consulta e retorna um único objeto JSON contendo os dados solicitados. Esse modelo flexível permite que o cliente tenha controle sobre a estrutura da resposta, reduzindo a quantidade de dados transferidos e melhorando a eficiência da comunicação.

Um dos principais benefícios do GraphQL é sua capacidade de consolidar múltiplas solicitações em uma única consulta. Em APIs REST, obter dados de várias fontes geralmente requer várias requisições, o que pode aumentar a latência e o uso de largura de banda. Com GraphQL, um único endpoint pode fornecer todos os dados necessários, pois a consulta pode navegar através de relações de dados de forma eficiente, reduzindo a necessidade de múltiplas chamadas de rede.

Além disso, GraphQL proporciona um robusto mecanismo de tipagem estática que define os tipos de dados e as relações entre eles. Esse esquema tipado permite que pessoas desenvolvedoras tenham uma compreensão clara dos dados disponíveis e das possíveis consultas, facilitando o desenvolvimento e a manutenção da API. Ferramentas como GraphiQL, um IDE interativo para explorar e testar consultas GraphQL, aproveitam esse esquema para fornecer uma experiência de desenvolvimento mais produtiva.

Apesar das muitas vantagens, a adoção do GraphQL também apresenta seus desafios. A complexidade do servidor GraphQL pode ser significativamente maior, pois ele precisa ser capaz de resolver consultas complexas de maneira eficiente. Além disso, a flexibilidade de GraphQL pode levar a consultas excessivamente complexas que sobrecarregam o servidor, exigindo a implementação de medidas de controle de profundidade e complexidade das consultas para garantir a performance.

Outro aspecto importante é a segurança. Como GraphQL permite consultas muito específicas e complexas, existe o risco de ataques de negação de serviço (DoS) através de consultas maliciosas. Portanto, é crucial implementar controles e validações adequadas para proteger a API contra tais ameaças.

Em comparação com REST, GraphQL oferece uma abordagem mais moderna e eficiente para a comunicação entre cliente e servidor. Enquanto REST segue uma filosofia de endpoints específicos para diferentes recursos, GraphQL centraliza a API em torno de um único endpoint e permite que o cliente especifique a estrutura dos dados. Isso resulta em maior flexibilidade e eficiência, especialmente em aplicações com necessidades complexas de dados e múltiplas fontes de dados.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, abordamos a criação de uma API REST com Spring MVC, passando pela construção dos componentes essenciais, como controllers, services e repositories. Utilizamos bibliotecas importantes, como Spring Data para a manipulação eficiente de dados e Spring Validation para garantir a consistência dos dados.

Discutimos também o retorno dos códigos de status HTTP corretos e a implementação de tratativas de erros de maneira eficaz. Configuramos o banco de dados em memória H2 para uma rápida prototipação e testes. Cada conceito foi ilustrado com exemplos de código, proporcionando uma compreensão prática de como construir uma API REST robusta e bem estruturada. Ao final, você adquiriu conhecimentos essenciais para o desenvolvimento de APIs modernas e escaláveis, podendo aplicar essas técnicas em projetos reais.

REFERÊNCIAS

AWS. **O que é uma API (interface de programação de aplicações)?**. 2024. Disponível em: <<https://aws.amazon.com/pt/what-is/api/>>. Acesso em: 05 ago. 2024.

Mozilla. **Cabeçalhos HTTP**. 2024. Disponível em: <<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Headers>>. Acesso em: 05 ago. 2024.

Mozilla. **Códigos de status de respostas HTTP**. 2024. Disponível em: <<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status>>. Acesso em: 05 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: API. DD. SPRING BOOT. MVC.

EMENDAS



POSTECH